Week09 – Timer Counter, Line Follower Robot
PC ____ รหัส _____ ชื่อ-สกุล _____

1/8: การทดลอง

1. MCS-51 (A)

2. MCS-51 (B)

3. ESP-32 (A)

4. ESP-32 (B)

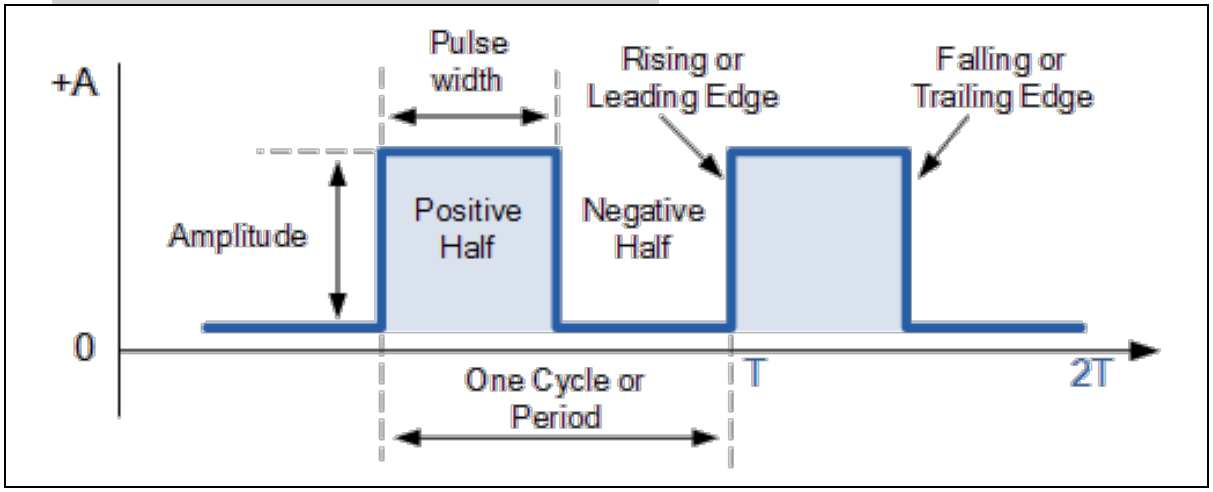
5. Node-RED (A)
   - จาก Mission-5: From ESP-32 to MQTT Broker and If xxx then LINE Alert
   - หากอุณหภูมิ เกิน xxxx หรือ ความชื้น เกิน yyyy ให้มีการแจ้งเตือนผ่าน LINE

6. Node-RED (B)
   - จาก Mission-6: From ESP-32 to MQTT Broker and Show on Node-RED Dashboard
   - ให้แสดงอุณหภูมิและความชื้นจาก DHT-22 ที่ Node-RED Dashboard
   - และสามารถควบคุม LED ได้ 2 ตัว

MCS-51	MCS-51	ESP-32	ESP-32	Node-RED If xxx then LINE	Node-RED Dashboard

2/8: เนื้อหา

1. Frequency and pulse width on Arduino



- การสั่งความถี่
  a. tone(pin, frequency)
  b. tone(pin, frequency, duration)
  c. https://www.arduitronics.com/article/20/arduino-sing-a-song
- การอ่านความถี่
  a. FreqMeasure.begin();
  b. FreqMeasure.available();
  c. FreqMeasure.read();
  d. FreqMeasure.end();
  e. http://www.ec.in.th/index.php?route=cms/article&article_id=36
- การสั่งความกว้าง
  a. analogWrite()
  b. https://www.arduino.cc/en/Tutorial/PWM
  c. https://www.arduino.cc/en/Tutorial/SecretsOfArduinoPWM
  d. Servo.attach(pin,min,max)
  e. servo.writeMicroseconds(uS)
  f. www.thaieasyelec.com/article-wiki/review-product-article/บทความตัวอย่างการควบคุม-rc-servo-motor-ด้วย-arduino.html
- การอ่านความกว้าง
  a. pulseIn()
  b. https://www.arduino.cc/en/Reference/PulseIn

2. Timer/Counter on MCS-51

TMOD Register <Byte Access>

(MSB) (LSB)

GATE	C/T	M1	M0	GATE	C/T	M1	M0
Timer 1				Timer 0			

GATE Gating control when set. The timer/counter is enabled only while the INTx pin is high and the TRx control pin is set. When cleared, the timer is enabled whenever the TRx control bit is set.

C/T Timer or counter selected cleared for timer operation (input from internal system clock). Set for counter operation (input from Tx input pin).

M1 Mode bit 1

M0 Mode bit 0

M1	M0	Mode	Operating Mode
0	0	0	13-bit timer mode 8-bit timer/counter THx with TLx as 5-bit prescaler
0	1	1	16-bit timer mode 16-bit timer/counters THx and TLx are cascaded; there is no prescaler
1	0	2	8-bit auto reload 8-bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows.
1	1	3	Split timer mode

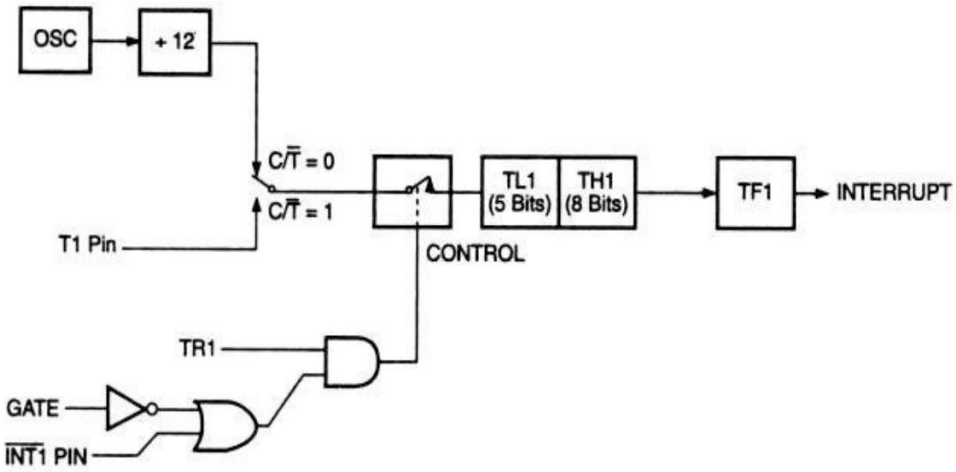
TCON Register <Bit Access>

(MSB) (LSB)

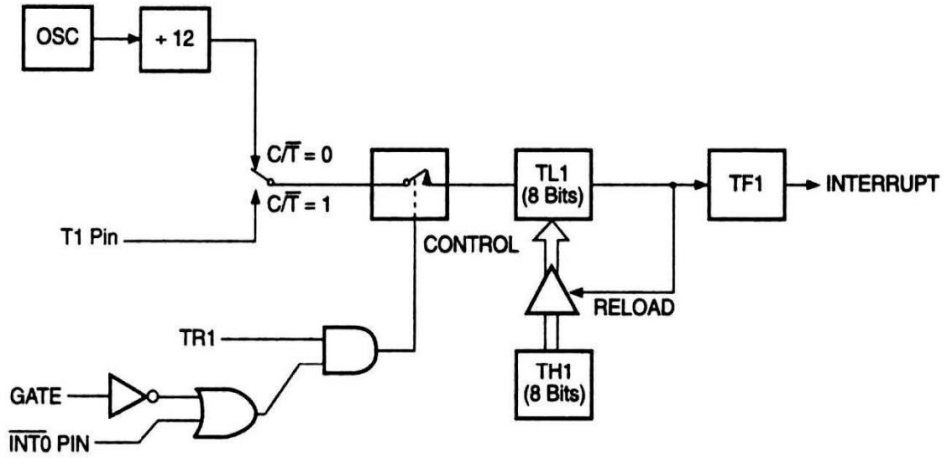
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

Symbol	Position	Name and Significance
TF1	TCON.7	Timer 1 overflow Flag. Set by hardware on timer/counter overflow. Cleared by hardware when processor vectors to interrupt routine.
TR1	TCON.6	Timer 1 Run control bit. Set/cleared by software to turn timer/counter on/off.
TF0	TCON.5	Timer 0 overflow Flag. Set by hardware on timer/counter overflow. Cleared by hardware when processor vectors to interrupt routine.
TR0	TCON.4	Timer 0 Run control bit. Set/cleared by software to turn timer/counter on/off.
IE1	TCON.3	Interrupt 1 Edge flag. Set by hardware when external interrupt edge detected. Cleared when interrupt processed.
IT1	TCON.2	Interrupt 1 Type control bit. Set/cleared by software to specify falling edge/low level triggered external interrupts.
IE0	TCON.1	Interrupt 0 Edge flag. Set by hardware when external interrupt edge detected. Cleared when interrupt processed.
IT0	TCON.0	Interrupt 0 Type control bit. Set/cleared by software to specify falling edge/low level triggered external interrupts.

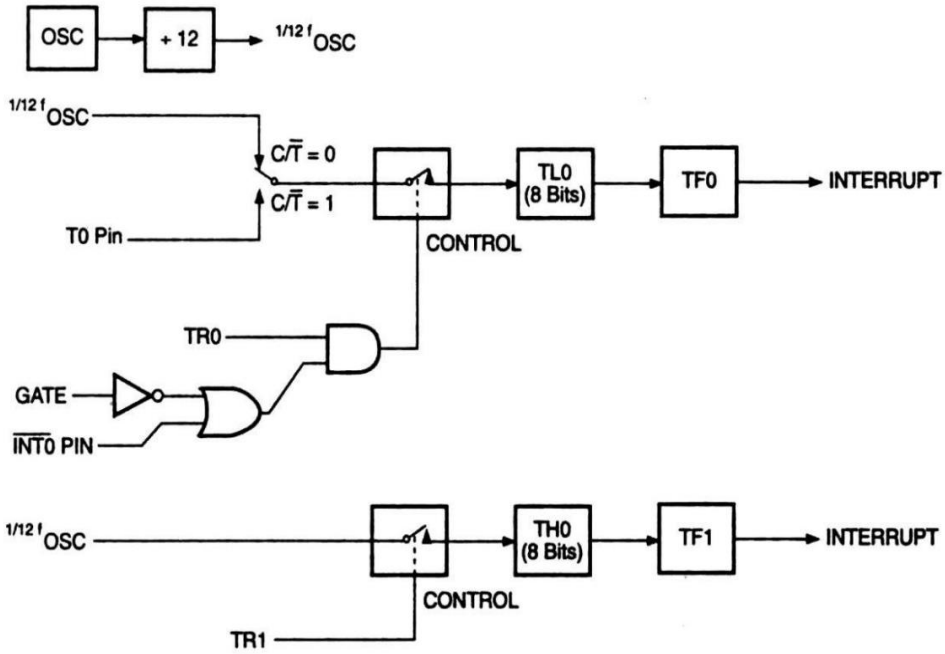
Mode 0: 13Bit , Mode 1: 16 Bit



Mode 2: 8Bit Auto Reload



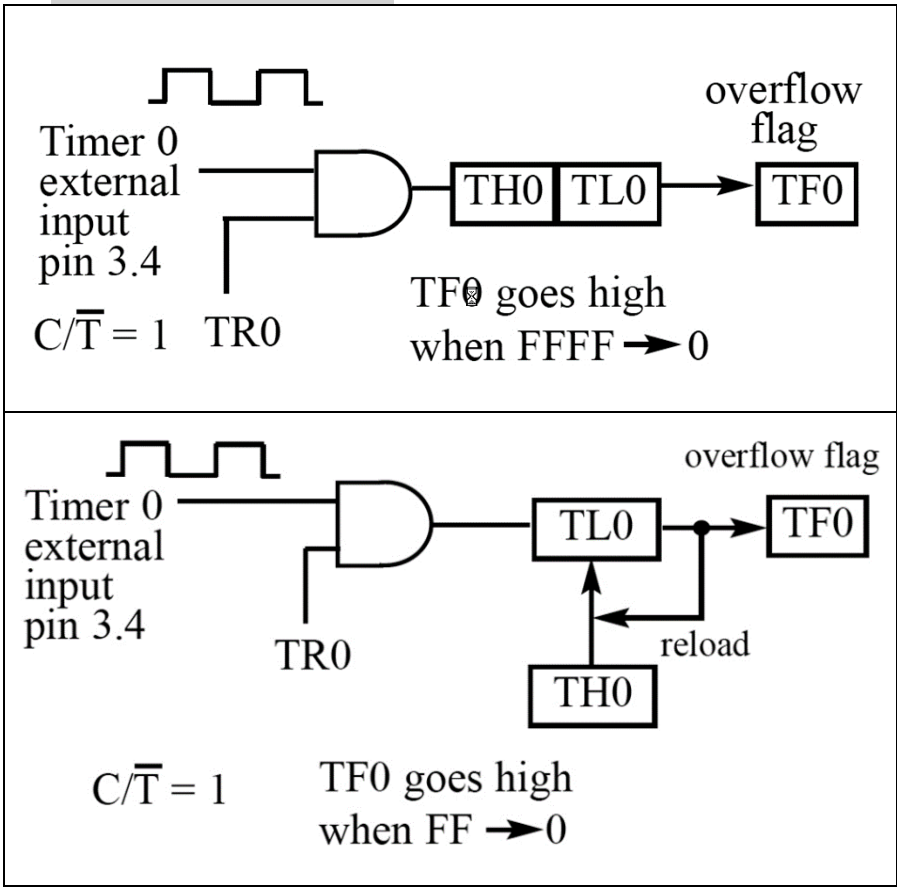
Mode 3: Split 8bit Time and 8Bit Counter



3. รีจีสเตอร์ที่สำคัญเกี่ยวกับ Timer/Counter

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMOD	**G(T1)**	**C/T**	**M1**	**M0**	**G(T0)**	**C/T**	**M1**	**M0**
TCON	**TF1**	**TR1**	**TF0**	**TR0**	**IE1**	**IT1**	**IE0**	**IT0**
SCON	**SM0**	**SM1**	**SM2**	**REN**	**TB8**	**RB8**	**TI**	**RI**
PCON	**SMOD1**	**SMOD0**	**-**	**POF**	**GF1**	**GF0**	**PD**	**IDL**
T2CON	**TF2**	**EXF2**	**RCLK**	**TCLK**	**EXEN2**	**TR2**	**C/T2**	**CP/RL2**

4. Counter Configuration



Counter 0 Mode1

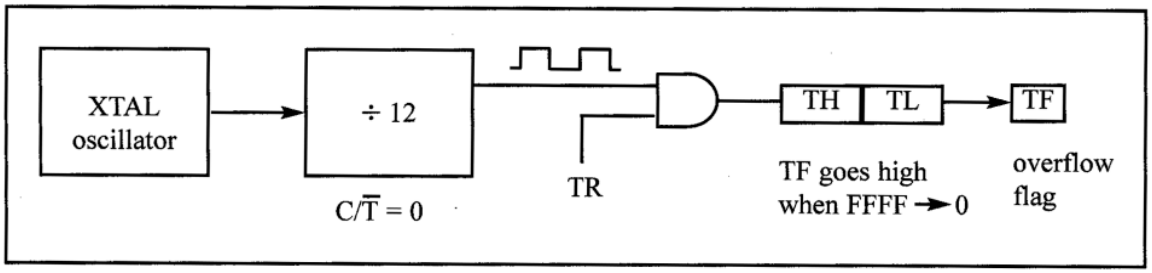
Counter 0 Mode2

<u>ตัวอย่าง</u> การใช้ในโหมด Counter

```
        MOV   TMOD,#00000101B      ; Using Counter Mode 1 16 Bit
        MOV   TH0,#0               ; Initial Value = 0000
        MOV   TL0,#0
        SETB  TR0                  ; Start Counter

Loop:   {Show TH0TL0}              ; Loop for Show Value of TH0TL0
        JMP   Loop
```

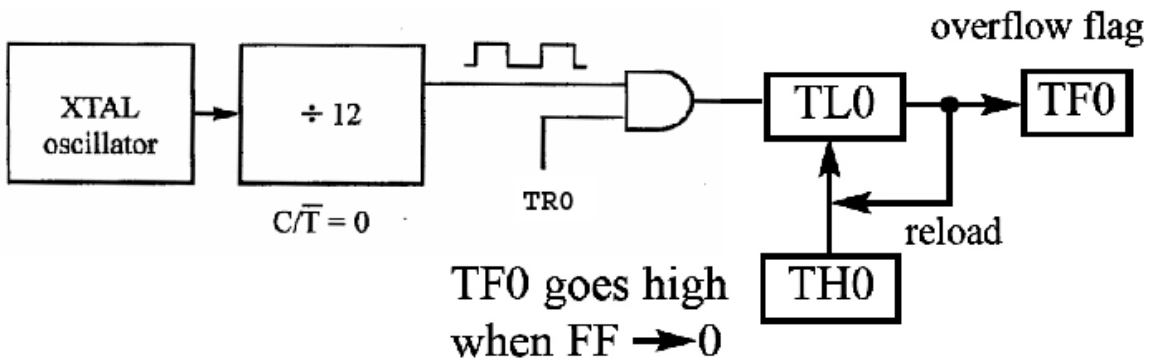
5. Timer Mode1 Configuration

Steps to program in mode 1	
1. Load the TMOD value	MOV TMOD,#00000001B
2. Load registers TL and TH	LOOP: MOV TH0,#0F2H
	MOV TL0,#0FFH
3. Start the timer (SETB TR0 or SETB TR1)	SETB TR0
< Work >	CPL P1
4. Keep monitoring the timer flag (TF) with the "JNB TFx,$"	JNB TF0,$
5. Stop the timer (CLR TR0 or CLR TR1)	CLR TR0
6. Clear the TF flag	CLR TF0
7. Go back to step 2	JMP LOOP



6. Timer Mode2 Configuration

Steps to program in mode 2	
1. Load the TMOD value	MOV TMOD,#20H
2. Load the TH registers	MOV TH1,#05H
3. Start the timer	SETB TR1
4. Keep monitoring the timer flag (TF)	LOOP: JNB TF0,$
< Work >	CPL P1
5. Clear the TF flag	CLR TF0
6. Go back to step 4	JMP LOOP



<u>ตัวอย่าง</u>.1 การใช้ Timer ในการหน่วงเวลา

เช่น ต้องการสร้างความถี่ 10 kHz → Period =0.1mSec → Period/2 = 50uSec
Delay 50uSec ที่บอร์ด Crystal 11.0592MHz

→ 1MC = 12/11.0592 = 1.085uSec

→ 50uSec = 46.08 MC

```
; Load Timer0 = 0–46 = -46 = FFD2H
MOV   TMOD,#01    ; Put Timer 0 in 16-bit mode
MOV   TH0,#0FFH   ; High byte of –46(FFD2H)
MOV   TL0,#0D2H   ; Low byte of –46(FFD2H)
SETB  TR0         ; Make Timer 0 Start Counting
CLR   TF0
JNB   TF0, $      ; If TF0 is not set, jump back to this same
                  ;  instruction
```

ตัวอย่าง.2 การใช้ Timer ในการหน่วงเวลาสำหรับ Serial Communication

Using Timer1, Mode2

http://www.intel.com/design/mcs51/applnots/270490.htm

Serial Mode1,3 และใช้ Timer 1 Mode2 →

$$\text{BaudRate} = \frac{2^{SMOD1} \times \text{Oscillater Frequency}}{32 \times 12 \times [256 - (th1)]}$$

9600 @18.432MHz → SMOD1=0, TH1=251(FBH)

9600 @11.0592MHz → SMOD1=0, TH1=253(FDH)

```
MOV   SCON,#050H        ; Serial Port Mode 3
ANL   PCON,#7FH         ; SMOD 1=2
MOV   TMOD,#20H         ; Timer1 Mode2
MOV   TH1,#0FDH         ; Reload value for desired baud
SETB  TR1               ; Turn on Timer1
```

Using Timer2

$$\text{Buad rate} = \frac{\text{Oscillater Frequency}}{32 \times [65536 - (RCAP2H,RCAP2L)]}$$

9600 @18.432MHz → RCAP2H, RCAP2L = 65476 = FFC4H

9600 @11.0592MHz → RCAP2H, RCAP2L = 65500 = FFDCH

```
MOV   SCON,#01010000B       ; Mode1 Serial Port
MOV   RCAP2H,#0FFH          ; Reload values for desired
MOV   RCAP2L,#0DCH          ; baud rate 9600@11.0592
MOV   T2CON,#00110100B      ; Timer 2 as baud rate generator
                            ; and turn on Timer 2
```

ตัวอย่าง.3 *Timer.C* <Ride-51> polling Delay

```
//*********************************************************//
// XTAL : 18.4322 MHz
// 1 MC = 12/18.432MHz = 0.651041667 uSec
// 1000 uSec = 1 mSec = 1536 MC --> 1536
// 65530-1536 = 64000 = 0xFA00
//*********************************************************//

#include <REG51.H>
void dmsec(unsigned char time)
   {    unsigned char i;
        for (i=0;i<time;i++)
        {   TH0 = 0xFA;
            TL0 = 0x00+18;          // +18 For move & compare instruction
            TF0 = 0;
            TR0 = 1;
            while(TF0 == 0);
            TR0 = 0;
        }
   }

void main()                         //********* Main
   {    TMOD = 0x01;                //Timer 16 bit
        while(1)
        {   P2 ^= 0x0ff;
            dmsec(50);              // Delay 50mS = 1/(2x50mS) = 10.00Hz
        }
   }
```

ตัวอย่าง.4 *Timer.C* <Ride-51> interrupts Delay

```
//*********************************************************//
// XTAL : 18.4322 MHz
// 1 MC = 12/18.432MHz = 0.651041667 uSec
// 1000 uSec = 1 mSec = 1536 MC --> 1536
// 65530-1536 = 64000 = 0xFA00
//*********************************************************//

#include <REG51.H>
int const_Delay,count_Delay;
void Timer0_isr(void) interrupt 1 {
    TR0 = 0;
    TH0 = 0xFA;
    TL0 = 0x00+18;    // +18 For move & compare instruction
    TR0 = 1;
    count_Delay++;
    if (count_Delay>=const_Delay) {
        P2 ^= 0x0ff;
        count_Delay = 0;
    }
}


void main() {                   //********* Main
    const_Delay = 50; // Delay 50mS = 1/(2x50mS) = 10.00Hz
    TMOD = 0x01;      //Timer 16 bit
    ET0 = 1;
    TR0 = 1;
    EA = 1;
    while(1)    {
        }
    }
```

ตัวอย่าง.5 Timer.C <Ride-51> timer0 mode2 interrupts for scan display

```
#include <REG51.H>
#define PortO_DispData P0
sbit CS_Seg0 = P1 ^ 4;
sbit CS_Seg1 = P1 ^ 5;
code unsigned char CodeSeg[] =
{ 0x77, 0x11, 0x6D, 0x5D, 0x1B, 0x5E, 0x7E, 0x15,
  0x7F, 0x5F, 0x3F, 0x7A, 0x66, 0x79, 0x6E, 0x2E
};

unsigned int timerCount, Counter = 0;

void Delay(unsigned char msec)
{ unsigned char i, j;
  while (msec)
  { for (i = 1; i <= 250; i++) //not :
      for (j = 1; j <= 113; j++); //with ;
    msec--;
  }
}

void Timer0_isr(void) interrupt 1 {
        unsigned char Count;
        timerCount++;
        timerCount = timerCount % 2;
        if(timerCount==0){
                Count = Counter/10;
                CS_Seg0 = 0; CS_Seg1 = 1;
                PortO_DispData = ~CodeSeg[Count];
                }
        if(timerCount==1){
                Count = Counter%10;
                CS_Seg0 = 1; CS_Seg1 = 0;
                PortO_DispData = ~CodeSeg[Count];
                }
        }

void main() {
        TMOD = 0x02;            // Timer0 Mode2
        TH0 = 0x00;
        TL0 = 0x00;
        ET0 = 1;
        TR0 = 1;
        EA = 1;
        while(1) {
                Counter++;
                if(Counter>99)    Counter=0;
                Delay(25);
                }
        }
```

<u>ตัวอย่าง.6</u> Lab8_Timer.Asm

สร้างความถี่ 1 kHz	= 1000 MicroSec	
Duty Cycle 50% Ton=Toff	= 500 MicroSec	
	= 500*18.432/12 MC	12 Clock/MC
	= 768 MC	
	= 500*18.432/6 MC	x2 Mode = 6 Clock/MC
	= 1536 MC	

```
;==== Lab8_Timer.Asm ====
        ORG     0000H
        JMP     0100H

        ORG     0100H
        MOV     SP,#3FH
        CLR     EA
        MOV     TMOD,#00010001B

T_OFF:  CLR     TR1                 ; Stop Timer            4
        MOV     A,#00H              ; Out Pulse             5
        MOV     P2,A                ; Out P1                6
        MOV     DPTR,#-1536+13      ; Reload Data           8
        MOV     TH1,DPH             ; For TH1               9
        MOV     TL1,DPL             ; For TL1               10
        CLR     TF1                 ; Initial Start         11
        SETB    TR1                 ; Start Timer1          12
        JNB     TF1,$               ; Wait Until TF On      13
                                    ;                       14
T_ON:   CLR     TR1                 ; 1MC                   15
        MOV     A,#0FFH             ; 1MC                   16
        MOV     P2,A                ; 1MC                   17
        MOV     DPTR,#-1536+15      ; 2MC                   20
        MOV     TH1,DPH             ; 2MC                   21
        MOV     TL1,DPL             ; 2MC                   22
        CLR     TF1                 ; 1MC                   23
        SETB    TR1                 ; 1MC                   24
        JNB     TF1,$               ; 2MC                   25
                                    ;                       26
        JMP     T_OFF               ; 2MC                   27

        END
```

<u>ตัวอย่าง.7</u> Lab8_Counter1.Asm

โปรแกรมสำหรับอ่านค่าการนับสัญญาณพัลล์ที่ P3.4 โดยไม่ใช้เทคนิค Counter

```
;====  Lab8_Counter1.Asm ====

Port_7Seg       EQU     P0
P_SegLeft       EQU     P1.4
P_SegRight      EQU     P1.5
P_TestIn        EQU     P3.4
Counter         EQU     20H

                ORG     0000H
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA
                SETB    P_SegLeft
                CLR     P_SegRight
                SETB    P_TestIn            ; Input Port
                MOV     Counter,#0

Main_Loop:      CALL    Display
                JB      P_TestIn,$
                JNB     P_TestIn,$
                INC     Counter
                JMP     Main_Loop

Display:        MOV     A,Counter
                ANL     A,#0FH
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                CPL     A
                MOV     Port_7Seg,A
                RET

CODE_TABLE:     DB      77H, 11H, 6DH, 5DH
                DB      1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH
                DB      66H, 79H, 6EH, 2EH


Delay_XXX:      CLR     A
                MOV     B,#200
_Dly00:         DJNZ    Acc,_Dly00
                DJNZ    B,_Dly00
                RET

                END
```

<u>ตัวอย่าง.8</u> Lab8_Counter2.Asm

โปรแกรมสำหรับอ่านค่าการนับสัญญาณพัลล์ที่ P3.4 (T0) โดยใช้เทคนิค Counter

```
;====  Lab8_Counter2.Asm ====

Port_7Seg       EQU     P0
P_SegLeft       EQU     P1.4
P_SegRight      EQU     P1.5
P_TestIn        EQU     P3.4
Counter         EQU     20H

                ORG     0000H
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA
                SETB    P_SegLeft
                CLR     P_SegRight
                SETB    P_TestIn            ; Input Port
                MOV     TMOD,#00000101B
                MOV     TH0,#0
                MOV     TL0,#0
                SETB    TR0

Main_Loop:      MOV     Counter,TL0
                CALL    Display
                JMP     Main_Loop

Display:        MOV     A,Counter
                ANL     A,#0FH
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                CPL     A
                MOV     Port_7Seg,A
                RET

CODE_TABLE:     DB      77H, 11H, 6DH, 5DH
                DB      1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH
                DB      66H, 79H, 6EH, 2EH


Delay_XXX:      CLR     A
                MOV     B,#200
_Dly00:         DJNZ    Acc,_Dly00
                DJNZ    B,_Dly00
                RET

                END
```

ตัวอย่าง.9 Lab8_Counter3.Asm

โปรแกรมสำหรับอ่านค่าการนับสัญญาณพัลล์ที่ P3.4 (T0) โดยใช้เทคนิค Counter

```
;== Lab8_Counter3.Asm ,Long==

Port_7Seg       EQU     P0
P_SegLeft       EQU     P1.4
P_SegRight      EQU     P1.5
P_TestIn        EQU     P3.4
Counter         EQU     20H

                ORG     0000H
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA
                SETB    P_TestIn ; Input Port
                MOV     TMOD,#00000101B
                MOV     TH0,#0
                MOV     TL0,#0
                SETB    TR0

Main_Loop:      MOV     Counter,TL0
                CALL    Display
                JMP     Main_Loop

Display:        MOV     A,Counter
                SETB    P_SegLeft
                CLR     P_SegRight
                CALL    _DISP00
                MOV     A,Counter
                SWAP    A
                CLR     P_SegLeft
                SETB    P_SegRight
                CALL    _DISP00
                RET

_DISP00:        ANL     A,#0FH
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                CPL     A
                MOV     Port_7Seg,A
                CLR     A
                DJNZ    Acc,$
                RET

CODE_TABLE:     DB      77H, 11H, 6DH, 5DH
                DB      1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH
                DB      66H, 79H, 6EH, 2EH


Delay_XXX:      CLR     A
                MOV     B,#200
_Dly00:         DJNZ    Acc,_Dly00
                DJNZ    B,_Dly00
                RET

                END
```

```
;== Lab8_Counter3.Asm ,Short ==

Port_7Seg       EQU     P0
P_SegLeft       EQU     P1.4
P_SegRight      EQU     P1.5
P_TestIn        EQU     P3.4
Counter         EQU     20H

                ORG     0000H
                JMP     0100H

                ORG     0100H
                MOV     SP,#3FH
                CLR     EA
                SETB    P_TestIn ; Input Port
                MOV     TMOD,#00000101B
                MOV     TH0,#0
                MOV     TL0,#0
                SETB    TR0

Main_Loop:      MOV     Counter,TL0
                CALL    Display
                JMP     Main_Loop

Display:        MOV     A,Counter
                SETB    P_SegLeft
                CLR     P_SegRight
                CALL    _DISP00
                MOV     A,Counter
                SWAP    A
                CLR     P_SegLeft
                SETB    P_SegRight
_DISP00:        ANL     A,#0FH
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                CPL     A
                MOV     Port_7Seg,A
                CLR     A
                DJNZ    Acc,$
                RET


CODE_TABLE:     DB      77H, 11H, 6DH, 5DH
                DB      1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH
                DB      66H, 79H, 6EH, 2EH


Delay_XXX:      CLR     A
                MOV     B,#200
_Dly00:         DJNZ    Acc,_Dly00
                DJNZ    B,_Dly00
                RET

                END
```

<u>ตัวอย่าง.10</u> Timer Interrupts

โปรแกรมสแกนหน้าจอ 7Segment ด้วยการใช้ Timer Interrupts

```
Port_7Seg       EQU     P0
P_SegLeft       EQU     P1.4
P_SegRight      EQU     P1.5
Counter         EQU     20H
BitDispLR       EQU     21H.0

                ORG     0000H
                JMP     0100H

                ORG     000BH
                JMP     T0_ISR

                ORG     0100H
                CLR     EA
                MOV     SP,#3FH
                MOV     Counter,#00H
                MOV     TMOD,#02H       ; Timer0 Mode2
                MOV     TH0,#0
                MOV     TL0,#0
                SETB    ET0
                SETB    EA
                SETB    TR0

Main_Loop:      INC     Counter
                CALL    Delay
                JMP     Main_Loop

Delay:          MOV     R7,#20
                MOV     A,#00
                MOV     B,#00
_Dly00:         DJNZ    B,_Dly00
                DJNZ    Acc,_Dly00
                DJNZ    R7,_Dly00
                RET

T0_ISR:         PUSH    Acc
                PUSH    DPH
                PUSH    DPL
                MOV     A,Counter
                JB      BitDispLR,_DRight
_DLeft:         SETB    P_SegLeft
                CLR     P_SegRight
                JMP     _DShow
_DRight:        CLR     P_SegLeft
                SETB    P_SegRight
                SWAP    A
_DShow:         ANL     A,#0FH
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                CPL     A
                MOV     Port_7Seg,A
                CPL     BitDispLR
                POP     DPL
                POP     DPH
                POP     Acc
                RETI

CODE_TABLE:     DB      77H, 11H, 6DH, 5DH
                DB      1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH
                DB      66H, 79H, 6EH, 2EH

                END
```

<u>ตัวอย่าง.11</u> Timer2 Interrupt < Ref=http://web.mit.edu/6.115/www/document/8051.pdf >

T2CON: Timer/Counter 2 Control Register. Bit Addressable
8052 Only

TF2	EXF2	RCLK	TCLK	EXEN2	TR2	C/$\overline{T2}$	CP/$\overline{RL2}$

TF2	T2CON. 7	Timer 2 overflow flag set by hardware and cleared by software. TF2 cannot be set when either RCLK = 1 or CLK = 1
EXF2	T2CON. 6	Timer 2 external flag set when either a capture or reload is caused by a negative transition on T2EX, and EXEN2 = 1. When Timer 2 interrupt is enabled, EXF2 = 1 will cause the CPU to vector to the Timer 2 interrupt routine. EXF2 must be cleared by software.
RCLK	T2CON. 5	Receive clock flag. When set, causes the Serial Port to use Timer 2 overflow pulses for its receive clock in modes 1 & 3. RCLK = 0 causes Timer 1 overflow to be used for the receive clock.
TLCK	T2CON. 4	Transmit clock flag. When set, causes the Serial Port to use Timer 2 overflow pulses for its transmit clock in modes 1 & 3. TCLK = 0 causes Timer 1 overflows to be used for the transmit clock.
EXEN2	T2CON. 3	Timer 2 external enable flag. When set, allows a capture or reload to occur as a result of negative transition on T2EX if Timer 2 is not being used to clock the Serial Port. EXEN2 = 0 causes Timer 2 to ignore events at T2EX.
TR2	T2CON. 2	Software START/STOP control for Timer 2. A logic 1 starts the Timer.
C/$\overline{T2}$	T2CON. 1	Timer or Counter select. 0 = Internal Timer. 1 = External Event Counter (falling edge triggered).
CP/$\overline{RL2}$	T2CON. 0	Capture/Reload flag. When set, captures will occur on negative transitions at T2EX if EXEN2 = 1. When cleared, Auto-Reloads will occur either with Timer 2 overflows or negative transitions at T2EX when EXEN2 = 1. When either RCLK = 1 or TCLK = 1, this bit is ignored and the Timer is forced to Auto-Reload on Timer 2 overflow.

Timer/Counter 2 Setup

Except for the baud rate generator mode, the values given for T2CON do not include the setting of the TR2 bit. Therefore, bit TR2 must be set, separately, to turn the Timer on.

As a Timer:

Table 7

MODE	T2CON	
	INTERNAL CONTROL (NOTE 1)	EXTERNAL CONTROL (NOTE 2)
16-bit Auto-Reload	00H	08H
16-bit Capture	01H	09H
BAUD rate generator receive & transmit same baud rate	34H	36H
receive only	24H	26H
transmit only	14H	16H

As a Counter:

Table 8

MODE	TMOD	
	INTERNAL CONTROL (NOTE 1)	EXTERNAL CONTROL (NOTE 2)
16-bit Auto-Reload	02H	0AH
16-bit Capture	03H	0BH

NOTES:
1. Capture/Reload occurs only on Timer/Counter overflow.
2. Capture/Reload occurs on Timer/Counter overflow and a 1 to 0 transition on T2EX (P1.1) pin except when Timer 2 is used in the baud rate generating mode.

8052 Register Byte Address

80	P0	SP	DPL	DPH				PCON	87
88	TCON	TMOD	TL0	TL1	TH0	TH1			8F
90	P1								97
98	SCON	SBUF							9F
A0	P2								A7
A8	IE								AF
B0	P3								B7
B8	IP								B9
C0									C7
C8	T2CON		RCAP2L	RCAP2H	TL2	TH2			CF
D0	PSW								D7
D8									DF
E0	ACC								E7
E8									EF
F0	B								F7
F8									FF

Blue background are I/O port SFRs
Yellow background are control SFRs
Green background are other SFRs

T2CON – SFR Bit Address

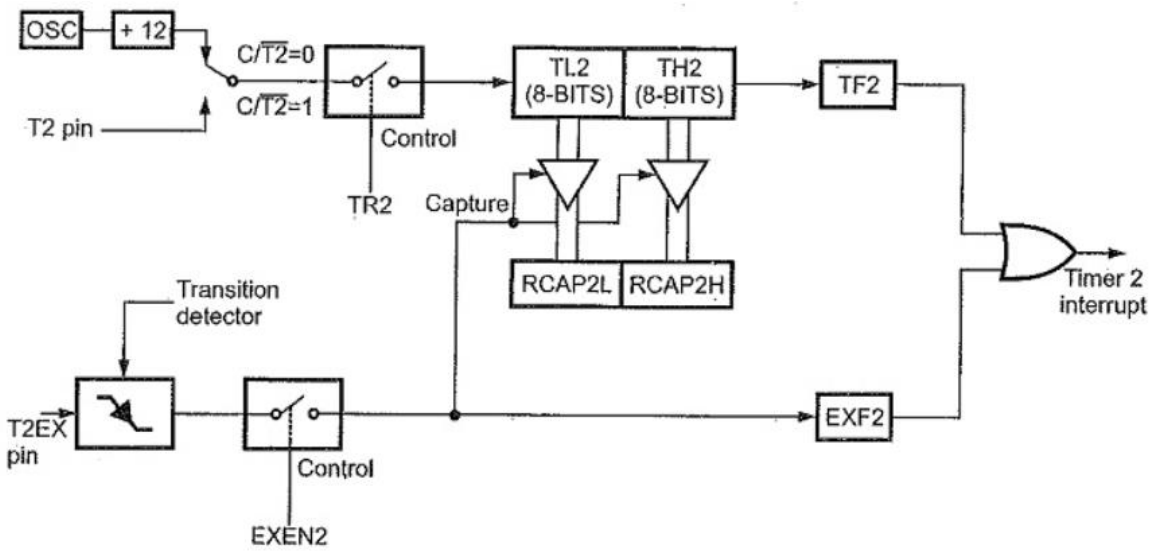
The operation of Timer 2 (T2) is controlled almost entirely by the T2CON SFR, at address C8h. Note that since this SFR is evenly divisible by 8 that it is bit-addressable.

BIT	NAME	BIT ADDRESS	DESCRIPTION
7	TF2	CFh	**Timer 2 Overflow**. This bit is set when T2 overflows. When T2 interrupt is enabled, this bit will cause the interrupt to be triggered. This bit will not be set if either TCLK or RCLK bits are set.
6	EXF2	CEh	**Timer 2 External Flag**. Set by a reload or capture caused by a 1-0 transition on T2EX (P1.1), but only when EXEN2 is set. When T2 interrupt is enabled, this bit will cause the interrupt to be triggered.
5	RCLK	CDh	**Timer 2 Receive Clock**. When this bit is set, Timer 2 will be used to determine the serial port receive baud rate. When clear, Timer 1 will be used.
4	TCLK	CCh	**Timer 2 Receive Clock**. When this bit is set, Timer 2 will be used to determine the serial port transmit baud rate. When clear, Timer 1 will be used.
3	EXEN2	CBh	**Timer 2 External Enable**. When set, a 1-0 transition on T2EX (P1.1) will cause a capture or reload to occur.
2	TR2	CAh	**Timer 2 Run**. When set, timer 2 will be turned on. Otherwise, it is turned off.
1	C/T2	C9h	**Timer 2 Counter/Interval Timer**. If clear, Timer 2 is an interval counter. If set, Timer 2 is incremented by 1-0 transition on T2 (P1.0).
0	CP/RL2	C8h	**Timer 2 Capture/Reload**. If clear, auto reload occurs on timer 2 overflow, or T2EX 1-0 transition if EXEN2 is set. If set, a capture will occur on a 1-0 transition of T2EX if EXEN2 is set.

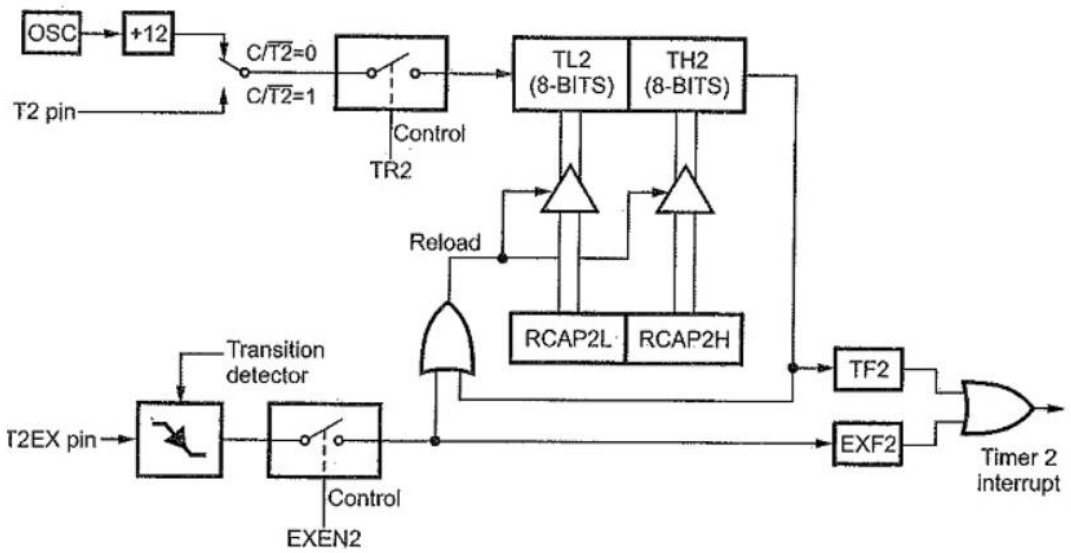
Timer 2 Mode Select

RCLK + TCLK	CP/$\overline{RL2}$	TR2	Mode
0	0	1	16-bit Auto-Reload
0	1	1	16-bit Capture
1	X	1	Baud Rate Generator
X	X	0	(Off)

Timer 2 – 16 Bit Capture Mode



Timer 2 – 16 Bit Auto Reload Mode



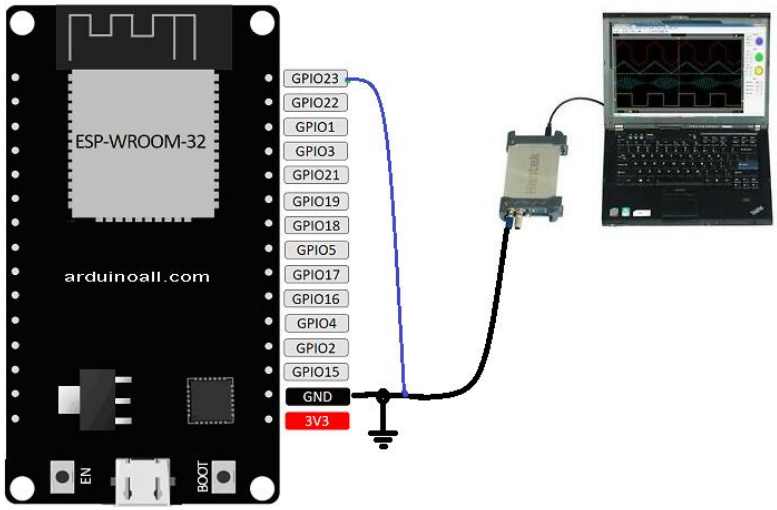

```
T2CON       EQU     0C8H
RCAP2L      EQU     0CAH
RCAP2H      EQU     0CBH
TL2         EQU     0CCH
TH2         EQU     0CDH
TR2         EQU     T2CON.2
TF2         EQU     T2CON.7
ET2         EQU     IE.5

Port_7Seg   EQU     P0
P_SegLeft   EQU     P1.4
P_SegRight  EQU     P1.5
Counter     EQU     20H
BitDispLR   EQU     21H.0

            ORG     0000H
            JMP     0100H
```

```
            ORG     002BH           ; Timer2 Interrupt Vector
            JMP     T2_ISR

            ORG     0100H
            CLR     EA
            MOV     SP,#3FH
            MOV     T2CON,#00       ; Timer 16Bit Auto-Reload
            MOV     DPTR,#0D000H    ; number of MC
            Mov     RCAP2H,DPH
            Mov     RCAP2L,DPL
            Mov     TH2,DPH
            Mov     TL2,DPL
            SETB    TR2             ; Start Timer2
            SETB    ET2             ; Enable Timer2
            SETB    EA              ; Enable All

Main_Loop:  DEC     Counter
            CALL    Delay
            JMP     Main_Loop

Delay:      MOV     R7,#20
            MOV     A,#00
            MOV     B,#00
_Dly00:     DJNZ    B,_Dly00
            DJNZ    Acc,_Dly00
            DJNZ    R7,_Dly00
            RET

T2_ISR:     PUSH    Acc             ; Save DPTR, Acc to Stack
            PUSH    DPH
            PUSH    DPL
            CLR     TF2             ; Clear Timer2 Overflow
            MOV     A,Counter
            JB      BitDispLR,_DRight
_DLeft:     SETB    P_SegLeft
            CLR     P_SegRight
            JMP     _DShow
_DRight:    CLR     P_SegLeft
            SETB    P_SegRight
            SWAP    A
_DShow:     ANL     A,#0FH
            MOV     DPTR,#CODE_TABLE
            MOVC    A,@A+DPTR
            CPL     A
            MOV     Port_7Seg,A
            CPL     BitDispLR
            POP     DPL             ; Get DPTR, Acc from Stack
            POP     DPH
            POP     Acc
            RETI                    ; Return from Interrupt

CODE_TABLE: DB      77H, 11H, 6DH, 5DH,1BH, 5EH, 7EH, 15H
            DB      7FH, 5FH, 3FH, 7AH, 66H, 79H, 6EH, 2EH

            END
```

3/8: MCS-51 VS ESP-32 in Timer/Counter

3A. Timer Delay >> สร้างสัญญาณ → จำนวน, เวลาOn, เวลาOff

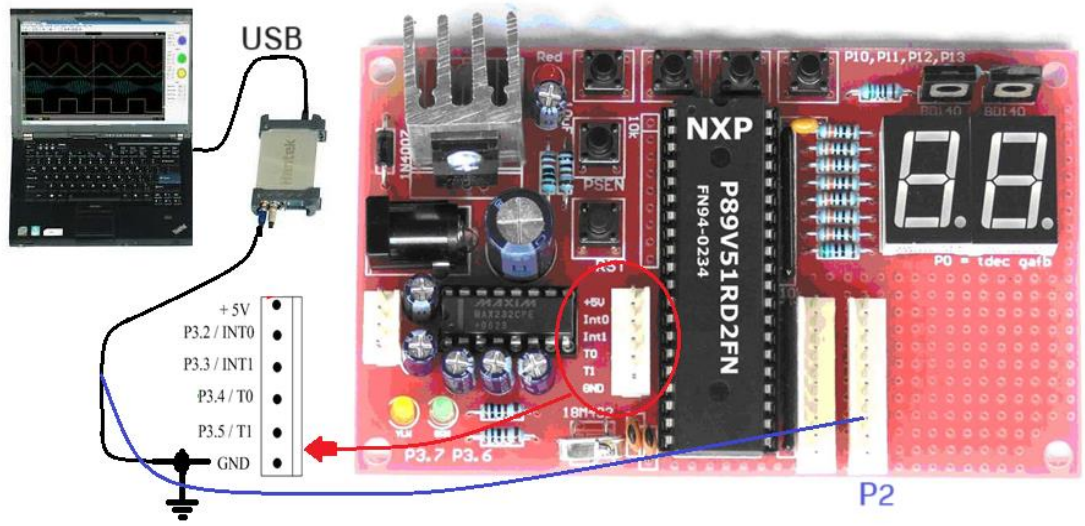
3a.1 ใช้ ESP32 สร้างความถี่ 1000 + รหัสสามตัวท้าย (±10) Hz หาก รหัสคู่กำหนด τ = 33% และ รหัสคี่กำหนด τ = 66%



ESP-32: Classic < 1,033 Hz >	ESP-32: Library < 1,033Hz >

int XX = 1000000 / 1033; // MicroSec
void setup() {
pinMode(23, OUTPUT);
}
 	
void loop() {
digitalWrite(23, HIGH);
delay(0.66 * XX);
digitalWrite(23, LOW);
delay(0.34 * XX);
}
	int ledPin = 23;
	int freq = 1033;
	int ledChannel = 0;
	int resolution = 8; // 255 Scale
	int dutyCycle = 255 * 0.66;
	
	void setup() {
	ledcSetup(ledChannel, freq, resolution);
	ledcAttachPin(ledPin, ledChannel);
	ledcWrite(ledChannel, dutyCycle);
	}
	
	void loop() {
	}

3a.2 จาก Lab8_Timer.Asm ให้สร้างความถี่ 1000 + รหัสสามตัวท้าย (±10) Hz รหัสคู่กำหนด τ = 33% และ รหัสคี่กำหนด τ = 66%



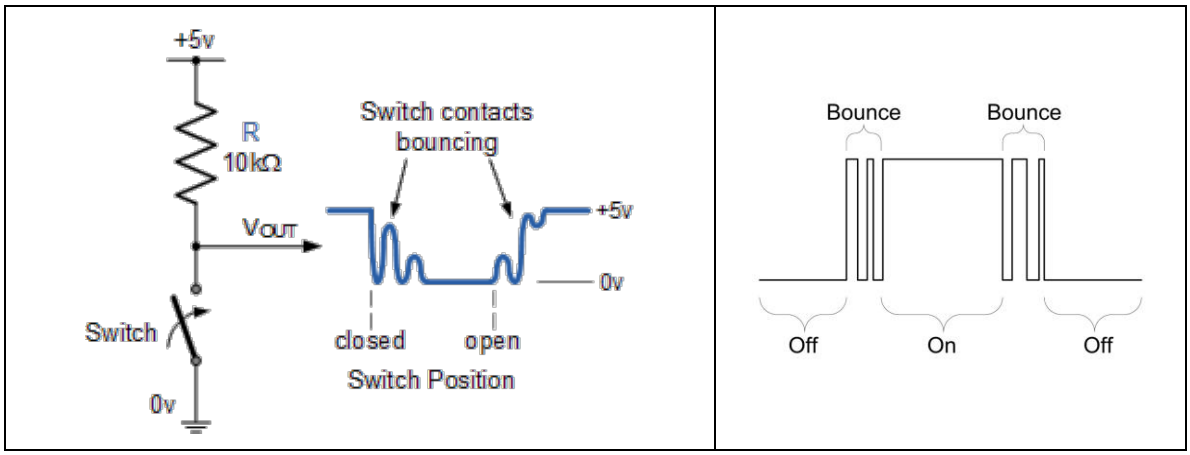
MCS-51

3B. Counter with Software >> อ่านสัญญาณ→ จำนวน, เวลาOn, เวลาOff

Debounce คืออะไร???

ก่อนจะทำความรู้จัก Debounce นั้นต้องทราบก่อนว่า Bounce คืออะไร

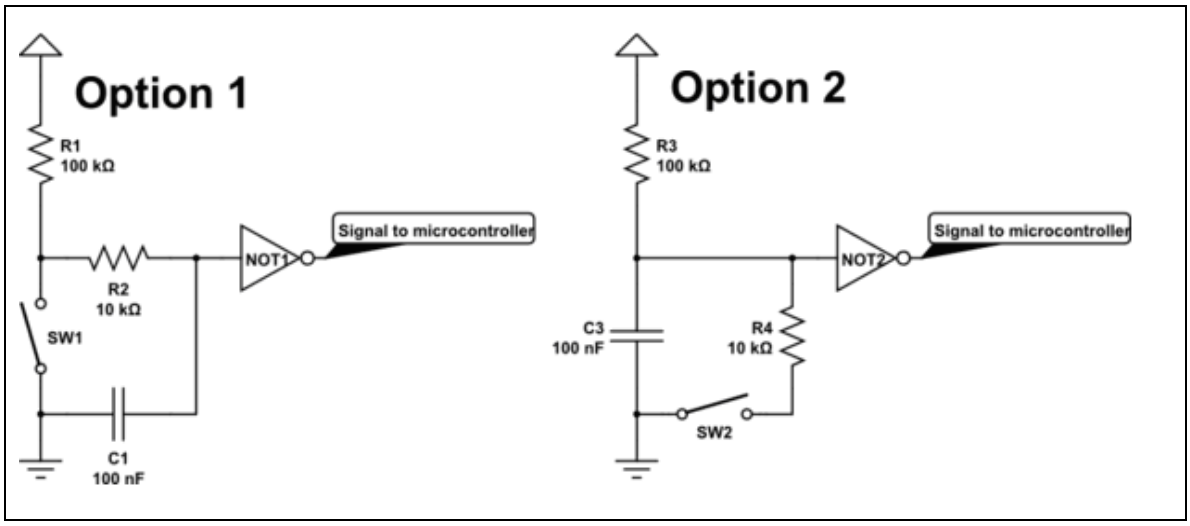
Bounce คือ สัญญาณที่เกิดขึ้นจากการที่การกดปุ่มแต่ละครั้ง เมื่อกดปุ่มหน้าสัมผัสที่ทำการสัมผัสกันนั้นยังไม่แนบสนิทดี จะทำให้เกิดสัญญาณวิ่งขึ้นวิ่งลง หรือ ก็คือ สัญญาณไม่นิ่ง ซึ่งจะทำให้ค่าที่ส่งออกไปนั้นคงาดเคลื่อน จึงต้องมีการทำ Debounce ขึ้นเพื่อป้องกันเหตุการณ์ที่จะเกิดขึ้นนี้



รูปภาพการเกิด Bouncing

จากรูปจะเห็นได้ว่าช่วง Switch Contact จะเกิดการ Bouncing ที่วงจรเกิดการ เปิด หรือ ปิด ซึ่ง หลักการทำงานของ Debounce คือ จะทำการนับลูกคลื่นของสัญญาณนั้นไปเรื่อยๆ จนสัญญาณนั้นเกิดการนิ่งแล้วซึ่งก็คือตรงกลางของรูปตัวอย่าง เพื่อค่าที่ส่งนั้นจะเกิดความถูกต้องไม่คลาดเคลื่อน

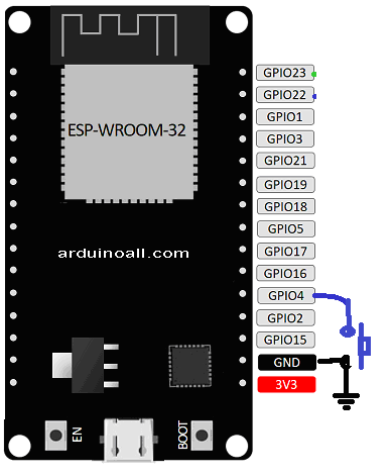
สามารถแก้ไขปัญหาการเกิด Bouncing ได้ด้วยการต่อวงจรภายนอก (Hardware Debounce) และการแก้ด้วยโปรแกรม (Software Debounce)



Hardware Debounce

3b.1 ใช้ ESP32 อ่านจำนวนครั้งการกดสวิตซ์ แสดงจำนวนครั้งที่ Serial Terminal

https://poundxi.com/tag/switch-debouncing

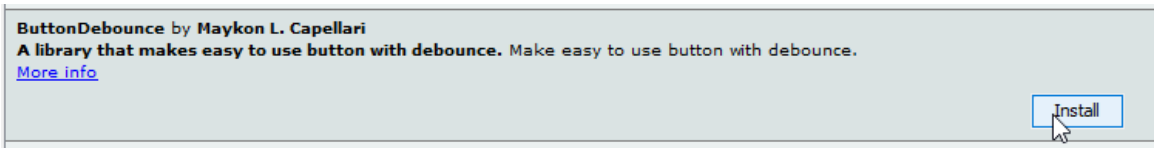


ESP-32: Classic

```
int InputPin = 4;
unsigned long int XXXX;

void setup() {
  pinMode(InputPin, INPUT_PULLUP);
  Serial.begin(115200);
  XXXX = 0;
}

void loop() {
  Serial.println(XXXX);
  while (digitalRead(InputPin) == 0);  delay(1);
  while (digitalRead(InputPin) == 1);  delay(1);
  XXXX++;
}
```

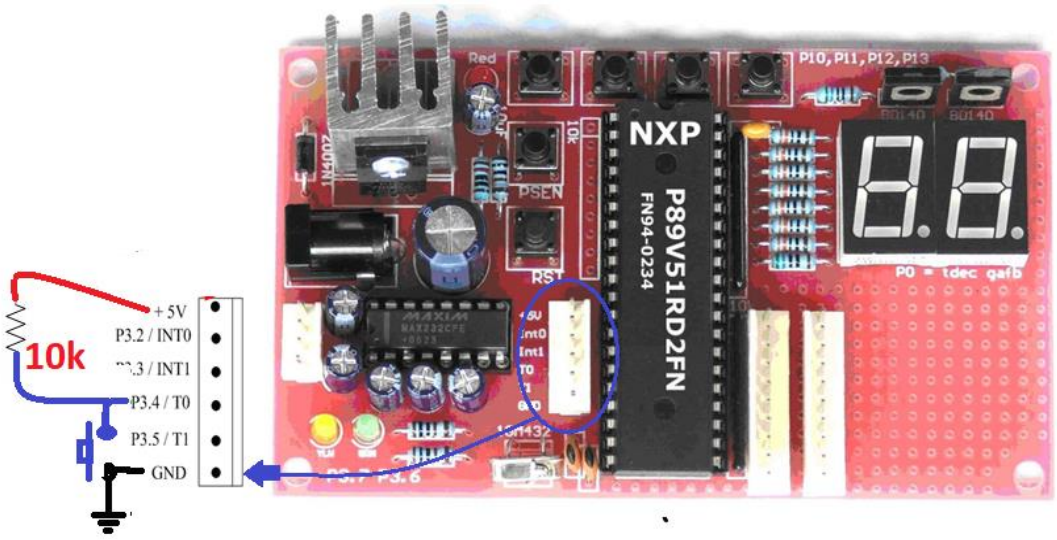
ESP-321: Library

Add Lib




```
#define pinInput 4
#define DebounceTimeMilliS 1
#include <ButtonDebounce.h>

ButtonDebounce button(pinInput, DebounceTimeMilliS);
unsigned long int XXXX;

void buttonChanged(int state) {
  if (button.state() == LOW)  Serial.println(++XXXX);
}

void setup() {
  Serial.begin(115200);
  button.setCallback(buttonChanged);
}

void loop() {
  button.update();
}
```

3b.2 จาก Lab8_Counter1.Asm อ่านจำนวนครั้งการกดสวิตซ์ แสดงผลที่ Left Segment

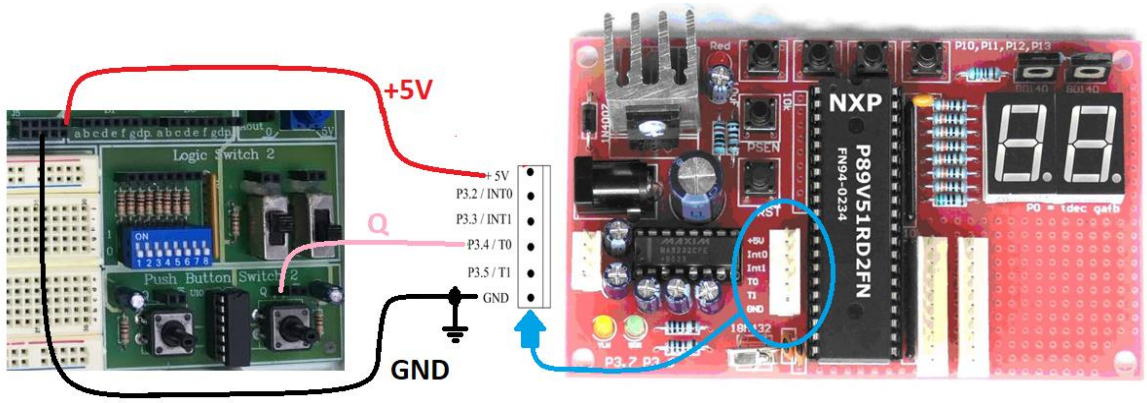


MCS-51

3C. Counter with Hardware >> อ่านสัญญาณ→ จำนวน, เวลาOn, เวลาOff

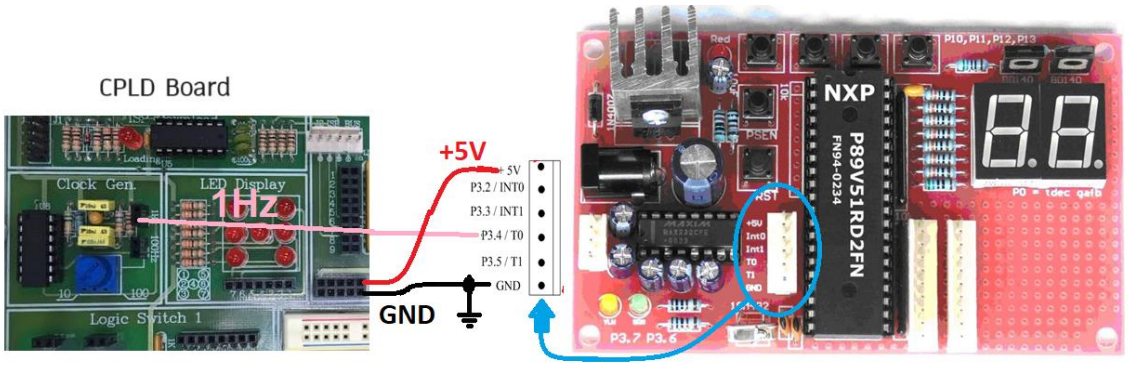
3C.1 ให้เขียนโปรแกรมตาม Lab8_Counter2.Asm เป็นโปรแกรมสำหรับการนับค่าด้วยการใช้ Counter0 เพื่อนับสัญญาณที่ขา P3.4(T0) ทดสอบด้วยการแตะสัญญาณ GND ที่ขา T0 สังเกตผลการนับที่ได้

- หากต้องการรับสัญญาณที่ขา T1 จะปรับปรุงแก้ไขโปรแกรมอย่างไร
- ระหว่างการป้อนอินพุต แบบข้อ 3b.2, ตามรูป Q, ตามรูปแต่เป็น Q-bar ผลการทำงานต่างกันอย่างไร เพราะเหตุใด



MCS-51

3C.2 ให้เขียนโปรแกรมตาม Lab8_Counter3.Asm เป็นโปรแกรมสำหรับการนับค่าด้วยการใช้ Counter0 เพื่อนับสัญญาณที่ขา P3.4(T0) และแสดงผลการนับ TH0-TL0 ที่ 7_Segment 2หลัก ทดสอบด้วยการป้อน Clock 1Hz จาก CPLD Board ที่ขา T0 สังเกตผลการนับที่ได้



MCS-51

3D. RIDE-51 C for MCS-51

```
//==============================================================
// Freq 1,033 Hz    ==> Period   = 968.0542 uSec
//                  ==> Duty 50% = 484.0271 uSec
// XTAL : 18.4322 MHz
// 1 MC = 12/18.432MHz = 0.651041667 uSec  --> Normal Mode
// 484.0271 uSec = 743.465634 MC --> 744 MC
// 65536-744 = 64792 = 0xFD18
//
// 1 MC = 6/18.432MHz = 0.3255208333 uSec  --> x2 Mode
// 484.0271 uSec = 1486.93127 MC --> 1487 MC
// 65536-1487 = 64049 = 0xFA31
//
//==============================================================
#include <REG51.H>
void main()                     // Main
   {  TMOD = 0x01;              // Timer 16 bit
      while(1)
      {  TR0 = 0;
         TH0 = 0xFA;
         TL0 = 0x31+12;         // +12 for move & compare instruction
         TF0 = 0;
         TR0 = 1;
         while(TF0 == 0);
         P2 ^= 0x0ff;
      }
   }
```

3d.1 Send Pulse Program using RIDE-C51

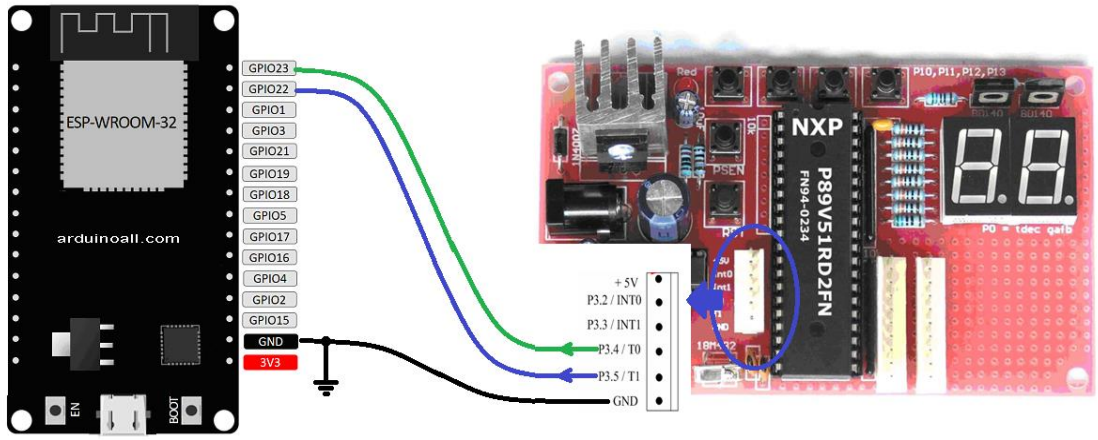
- ให้เขียนโปรแกรมด้วยภาษา C เพื่อรับค่าจาก P1 กำหนดให้ การทำงานตามตาราง
- ทดสอบการทำงานด้วยการป้อนอินพุตให้กับวงจรในข้อ 3C2
- หากส่ง Pulse ครบจำนวนที่กำหนดแล้ว ให้รอการกด Switch ใหม่ และให้ P3.2=High, 7Segment แสดง --

มีการกด Switch	จะทำให้	7_Seg แสดง	จำนวน Pulse ที่ส่งออกที่ P3.2	ระยะห่าง แต่ละ Pulse
SW0	P1.0=0	11	100	12 mSec
SW1	P1.1=0	33	300	12 mSec
SW2	P1.2=0	55	500	6 mSec
SW3	P1.3=0	77	700	6 mSec

MCS-51

3E. Pulse from MCS-51 to ESP-32

3E.1 ให้ MCS51 ส่ง pulse ออกที่ขา P3.4, P3.5 แล้วให้ ESP32 อ่านจำนวน Pulse ที่ส่งออกมา ให้แสดงค่าที่ได้ที่ Serial Terminal



ESP-32: Pulse Count

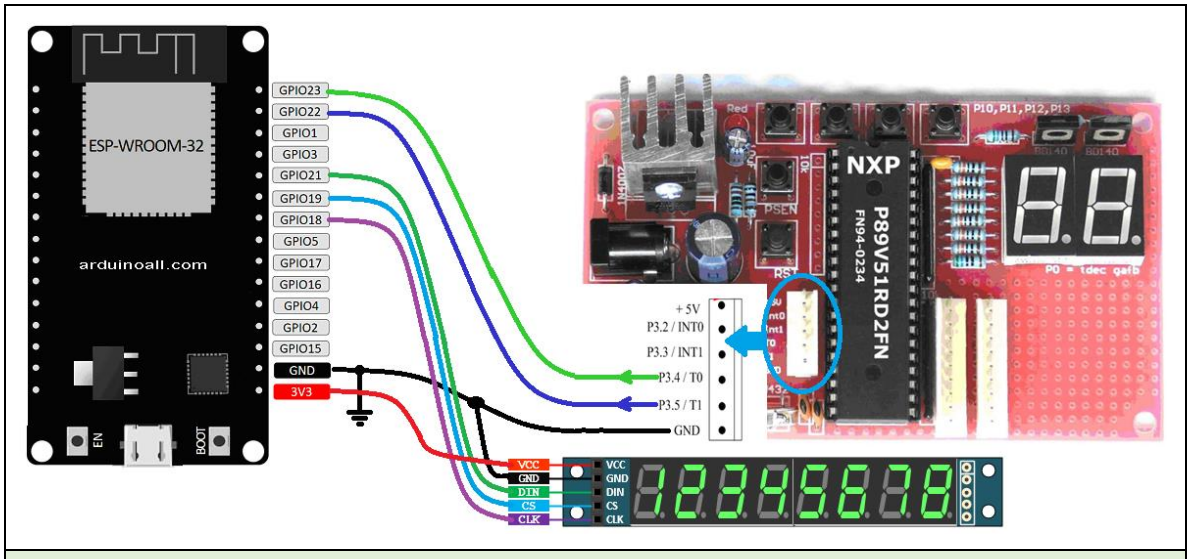
- Example: for 1 Input D23(ยังไม่สมบูรณ์ อยากได้ 2 Input D22, D23)>
- https://www.arduinoall.com/article/164/11-esp32-สอน-arduino-esp32-การใช้งาน-arduino-esp32-interrupt

```
int xPin = 23;
int mPin = 0;
unsigned long int XXXX = 0;

void setup() {
  pinMode(xPin, INPUT_PULLUP);
  pinMode(mPin, INPUT_PULLUP);
  Serial.begin(115200);
  Serial.println("\n\nPress Boot_Sw for Show Counter");
  XXXX = 0;
}

void loop() {
  while (digitalRead(xPin) == 1)  if (digitalRead(mPin) == 0)  Serial.println(XXXX);
  while (digitalRead(xPin) == 0)  if (digitalRead(mPin) == 0)  Serial.println(XXXX);
  XXXX++;
}
```

MCS-51: Pulse Out

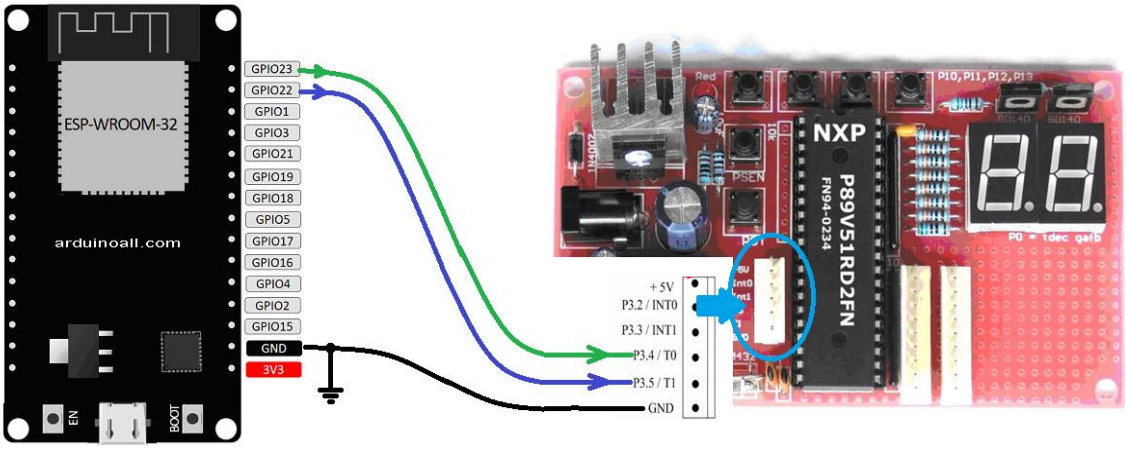
```
AP_DLY      EQU     30H
AN_DLY      EQU     31H
ANP_H       EQU     32H
ANP_L       EQU     33H
BP_DLY      EQU     34H
BN_DLY      EQU     35H
BNP_H       EQU     36H
BNP_L       EQU     37H

            ORG     0000H
            JMP     0100H

            ORG     0100H
            MOV     SP,#3FH
            CLR     EA
            MOV     AP_DLY,#12      ; delay Step
            MOV     DPTR,#-200      ; nPluse x2
            MOV     ANP_H,DPH
            MOV     ANP_L,DPL
            MOV     BP_DLY,#34      ; delay Step
            MOV     DPTR,#-600      ; nPluse x2
            MOV     BNP_H,DPH
            MOV     BNP_L,DPL

            MOV     AN_DLY,AP_DLY
            MOV     BN_DLY,BP_DLY
LOOP:       CALL    DLY
            CALL    PA_CHK
            CALL    PB_CHK
            JMP     LOOP

;==============================
PA_CHK:     MOV     A,ANP_H
            JNZ     _DC_NA
            MOV     A,ANP_L
            JNZ     _DC_NA
            RET
_DC_NA:     DEC     AN_DLY
            MOV     A,AN_DLY
            JNZ     _XPACK
            MOV     DPH,ANP_H
            MOV     DPL,ANP_L
            INC     DPTR
            MOV     ANP_H,DPH
            MOV     ANP_L,DPL
            CPL     P3.4
            MOV     AN_DLY,AP_DLY
_XPACK:     RET

;==============================
PB_CHK:     MOV     A,BNP_H
            JNZ     _DC_NB
            MOV     A,BNP_L
            JNZ     _DC_NB
            RET
_DC_NB:     DEC     BN_DLY
            MOV     A,BN_DLY
            JNZ     _XPBCK
            MOV     DPH,BNP_H
            MOV     DPL,BNP_L
            INC     DPTR
            MOV     BNP_H,DPH
            MOV     BNP_L,DPL
            CPL     P3.5
            MOV     BN_DLY,BP_DLY
_XPBCK:     RET


;==============================
DLY:        MOV     A,#80H
            DJNZ    Acc,$
            RET

            END
```

3E.2 จาก 3E.1 นอกจากแสดงค่าที่ได้ที่ Serial Terminal แล้วให้แสดงค่าที่ MAX7219 Display



ESP-32

3F. Pulse from ESP-32 to MCS-51



ให้ ESP32 ส่ง pulse ออกที่ขา 22, 23 แล้วให้ MCS51 อ่านจำนวน Pulse ที่ส่งออกมา

- ให้แสดงค่าที่ได้จากขา 22 จำนวน 2 หลักที่ 7Segment เมื่อกด SW_P1.0
- ให้แสดงค่าที่ได้จากขา 23 จำนวน 2 หลักที่ 7Segment เมื่อกด SW_P1.3

ESP-32

```
int Pin_A = 22; // Pin Assignment
int Pin_B = 23;

int nPulse_A = 0x123;        // number of pulse
int nPulse_B = 0x456;
int dPulse_A = 43;           // pulse delay (ms)
int dPulse_B = 21;

unsigned long int nCounter = 0;
int stsPulse_A = 0;
int stsPulse_B = 0;

void setup() {
  pinMode(Pin_A, OUTPUT);
  pinMode(Pin_B, OUTPUT);
  digitalWrite(Pin_A, HIGH);
  digitalWrite(Pin_B, HIGH);
  nPulse_A = nPulse_A * 2; // On and Off
  nPulse_B = nPulse_B * 2;
}

void loop() {
  delay(1);
  nCounter++;
  if (((nCounter % dPulse_A) == 0) && (nPulse_A >= 0))  {
    stsPulse_A = 1 - stsPulse_A;
    digitalWrite(Pin_A, stsPulse_A);
    nPulse_A--;
  }
  if (((nCounter % dPulse_B) == 0) && (nPulse_B >= 0))  {
    stsPulse_B = 1 - stsPulse_B;
    digitalWrite(Pin_B, stsPulse_B);
    nPulse_B--;
  }
}
```

MCS-51

```
T2CON       EQU     0C8H
TH2         EQU     0CDH
TL2         EQU     0CCH
RCAP2H      EQU     0CBH
RCAP2L      EQU     0CAH
TR2         EQU     T2CON.2
TF2         EQU     T2CON.7
ET2         EQU     IE.5


Port_7Seg   EQU     P0
P_SegLeft   EQU     P1.4
P_SegRight  EQU     P1.5
P_SW0       EQU     P1.0
P_SW3       EQU     P1.3
P_T0_34     EQU     P3.4
P_T1_35     EQU     P3.5
Counter     EQU     20H
BitDispLR   EQU     21H.0
BitShowAB   EQU     21H.1

            ORG     0000H
            JMP     0100H

            ORG     002BH
            JMP     T2_ISR

            ORG     0100H
            CLR     EA
            MOV     SP,#3FH
            SETB    P_SW0
            SETB    P_SW3
            SETB    P_T0_34
            SETB    P_T1_35

            MOV     TMOD,#01010101B
            MOV     TH0,#00
            MOV     TL0,#00
            MOV     TH1,#00
            MOV     TL1,#00
            SETB    TR0
            SETB    TR1

            MOV     T2CON,#00
                    ; Timer 16Bit Auto-Reload
            MOV     DPTR,#0D000H
            MOV     RCAP2H,DPH
            MOV     RCAP2L,DPL
            MOV     TH2,DPH
            MOV     TL2,DPL
            SETB    TR2         ; Start Timer2
            SETB    ET2         ; Enable Timer2
            SETB    EA          ; Enable All

            MOV     Counter,TL0


Main_Loop:  NOP
_Chk_SW0:   JB      P_SW0,_Chk_SW3
            SETB    BitShowAB

_Chk_SW3:   JB      P_SW3,_Chk_Show0
            CLR     BitShowAB

_Chk_Show0: JB      BitShowAB,_Chk_Show1
            MOV     Counter,TL0
            JMP     Main_Loop

_Chk_Show1: MOV     Counter,TL1
            JMP     Main_Loop

Delay:      MOV     R7,#20
            MOV     A,#00
            MOV     B,#00
_Dly00:     DJNZ    B,_Dly00
            DJNZ    Acc,_Dly00
            DJNZ    R7,_Dly00
            RET

T2_ISR:     PUSH    Acc
            PUSH    DPH
            PUSH    DPL
            CLR     TF2
            MOV     A,Counter
            JB      BitDispLR,_DRight
_DLeft:     SETB    P_SegLeft
            CLR     P_SegRight
            JMP     _DShow
_DRight:    CLR     P_SegLeft
            SETB    P_SegRight
            SWAP    A
_DShow:     ANL     A,#0FH
            MOV     DPTR,#CODE_TABLE
            MOVC    A,@A+DPTR
            CPL     A
            MOV     Port_7Seg,A
            CPL     BitDispLR
            POP     DPL
            POP     DPH
            POP     Acc
            RETI

CODE_TABLE: DB      77H, 11H, 6DH, 5DH
            DB      1BH, 5EH, 7EH, 15H
            DB      7FH, 5FH, 3FH, 7AH
            DB      66H, 79H, 6EH, 2EH

            END
```

4/8: ESP-32: RTOS

- https://www.iot.eng.kmutnb.ac.th/pub/docs/
- http://www.ayarafun.com/2016/12/basic-freertos-with-node32s-esp32/
- https://attaphon.medium.com/ลองเขียนโปรแกรมแบบ-multi-tasking-บน-esp32-กัน-rtos-fd098f798f8f
- https://www.hackster.io/luo-zeqing/implement-freertos-with-arduino-ide-on-esp32-444f54

ESP-32

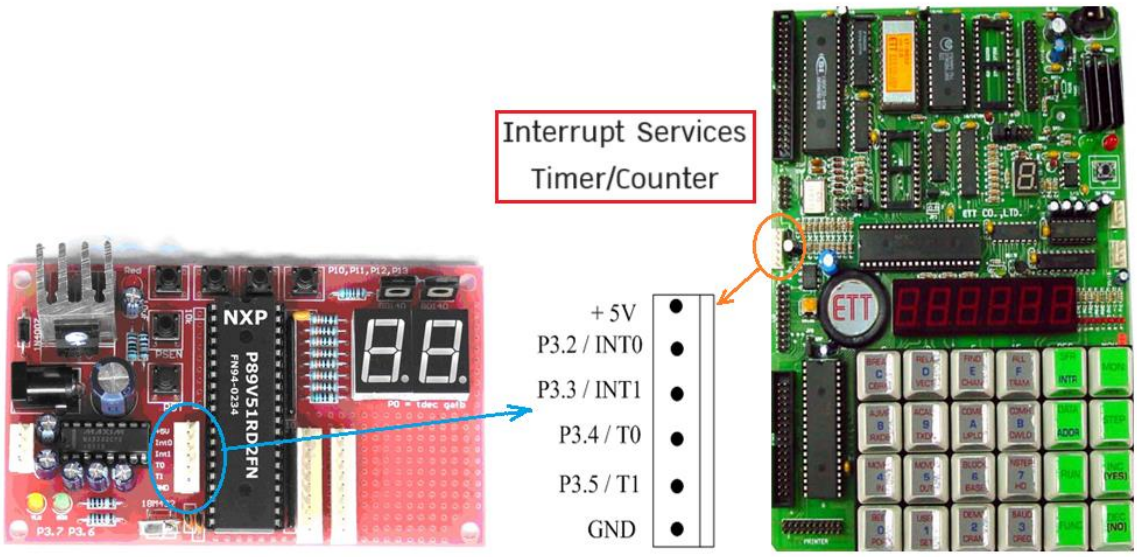
```
// ---------------------------------------------
#define BUILDIN_LED 2
#define ledPIN1 23
#define ledPIN2 22
int delayTask1 = 50, delayTask2 = 500; // mSec

// ---------------------------------------------
void first_task(void *pvParameter) {
  pinMode(ledPIN1, OUTPUT);
  while (true) {
    digitalWrite(ledPIN1, HIGH);
    vTaskDelay(delayTask1 / portTICK_RATE_MS);
    digitalWrite(ledPIN1, LOW);
    vTaskDelay(delayTask1 / portTICK_RATE_MS);
  }
}

// ---------------------------------------------
void second_task(void *pvParameter) {
  pinMode(ledPIN2, OUTPUT);
  while (true) {
    digitalWrite(ledPIN2, HIGH);
    vTaskDelay(delayTask2 / portTICK_RATE_MS);
    digitalWrite(ledPIN2, LOW);
    vTaskDelay(delayTask2 / portTICK_RATE_MS);
  }
}

// ---------------------------------------------
void setup() {
  Serial.begin(115200);
  pinMode(BUILDIN_LED, OUTPUT);
  // xTaskCreate(pvTaskCode, pcName, usStackDepth, pvParameters, uxPriority, pxCreatedTask)
  xTaskCreate(&first_task, "first_task", 1024, NULL, 5, NULL);
  xTaskCreate(&second_task, "second_task", 1024, NULL, 5, NULL);
}

// ---------------------------------------------
void loop() {
  if (Serial.available() > 0) {
    int temp = Serial.parseInt();
    Serial.println(temp);
    delayTask1 = (temp > 10 & temp < 20) ? 1000 / ((temp - 10) * 2) : delayTask1;
    delayTask2 = (temp > 20 & temp < 30) ? 1000 / ((temp - 20) * 2) : delayTask2;
  }
  digitalWrite(BUILDIN_LED, HIGH);
  delay(250);
  digitalWrite(BUILDIN_LED, LOW);
  delay(250);
}
```

5/8: ESP-32: EDUBlocks and MicroPython

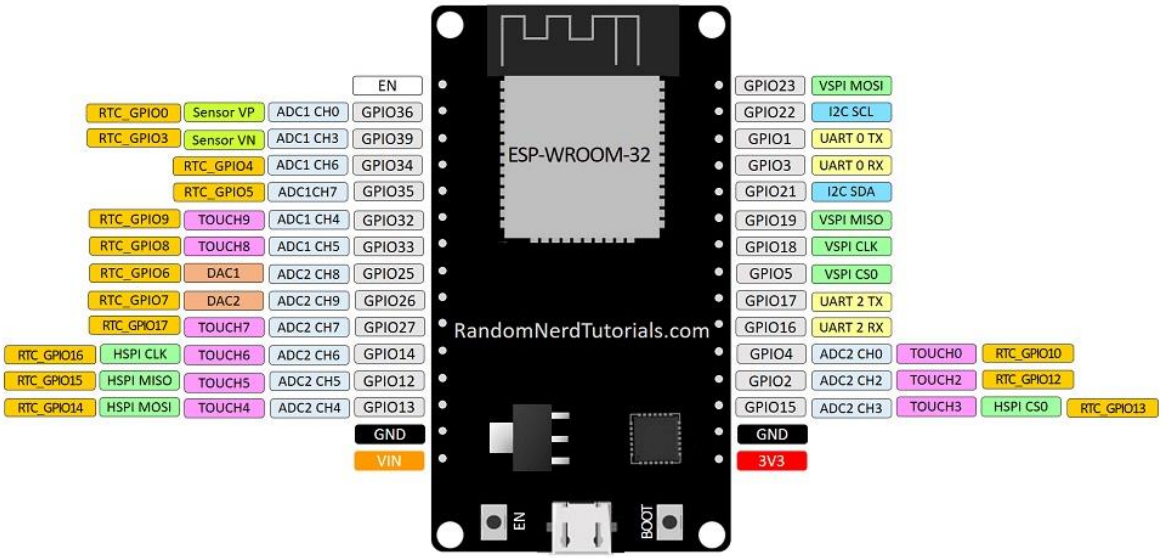
- https://www.electromaker.io/tutorial/blog/welcome-to-the-esp32-running-edublocks
- https://www.iot.eng.kmutnb.ac.th/pub/docs/2018/edublocks_esp32_micropython/

6/8: Hardware Reference

MCS-51: SUT V5 Board, ET-8032 V2 Board



ESP32 Devkit V1 – DOIT Version with 30 GPIOs



- https://www.sparkfun.com/tutorials/348
- http://forum.arduino.cc/index.php?topic=22060.0

7/8: Node-RED

Read More

- http://noderedguide.com/
- Install and First Flows
  - https://nodered.org/docs/tutorials/first-flow
  - https://www.youtube.com/watch?v=jqJ-D41SFL8
  - https://www.youtube.com/watch?v=rQuyjF2_Q4M
- Install https://jackrobotics.me/node-red-ep0-install-windows-1c836a715fe
- Get LINE Token https://jackrobotics.me/แจ้งเตือน-line-notify-เตือนง่ายได้ใจ-7328125083ec
- Data to LINE with node-red https://jackrobotics.me/จับ-line-notify-ยัด-node-red-โดยใช้ความสามารถเดิมของ-node-red-1730d83c067c

Experiment

1. Install Node-RED and Start
2. Mission-1: Creating your first flow
3. Mission-2: Node-RED Publish and Subscribe
4. Mission-3: LINE Test >> https://www.youtube.com/watch?v=2vGtXZRlGy8
5. Mission-4: LINE Bot with Node-RED >> https://youtu.be/4KKIEmWYCGQ?t=794
6. Mission-5: From ESP-32 to MQTT Broker and If xxx then LINE Alert
   - หากอุณหภูมิ เกิน xxxx หรือ ความชื้น เกิน yyyy ให้มีการแจ้งเตือนผ่าน LINE
7. Mission-6: From ESP-32 to MQTT Broker and Show on Node-RED Dashboard
   - แสดงอุณหภูมิและความชื้นจาก DHT-22 ที่ Node-RED Dashboard
   - ควบคุม LED 2 ตัว

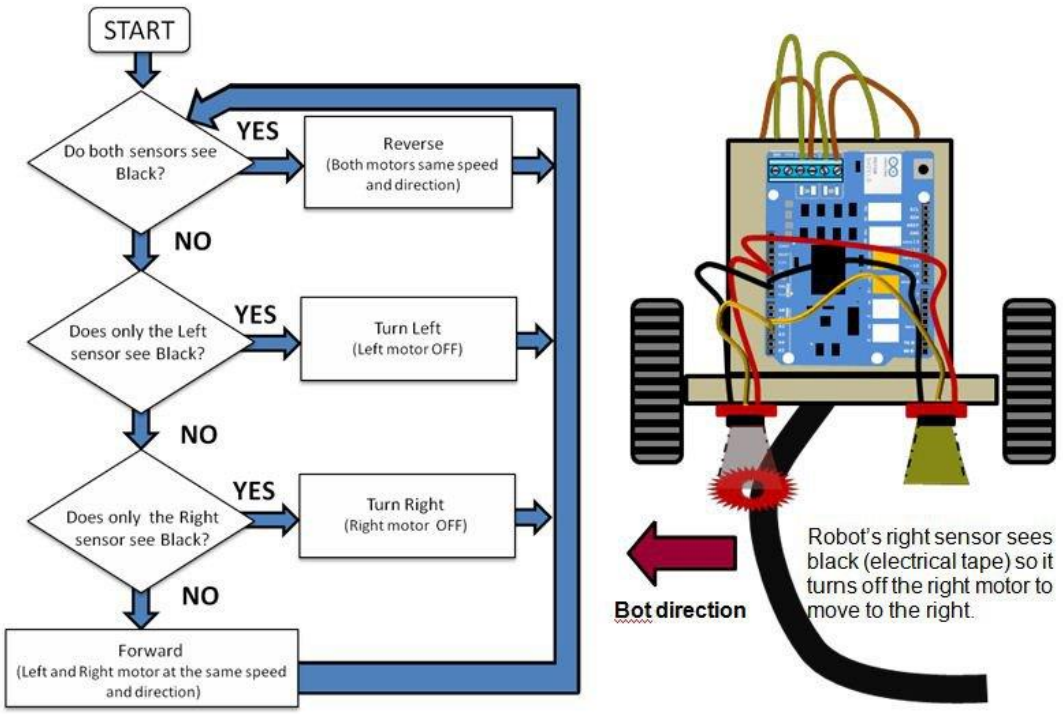
8/8: Line Follower Robot

Read More

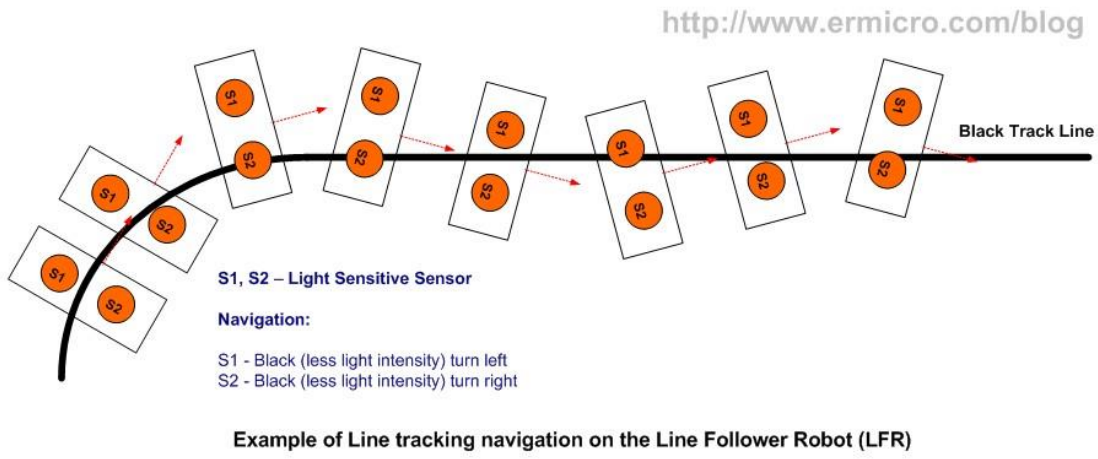
- https://www.cybertice.com/p/1449
- https://www.educba.com/line-follower-robot/

Experiment

1. การเขียนโปรแกรม



http://chipkit.net/arduino-mc-line-follower/



http://www.ermicro.com/blog/?p=1097